

Progetto 2

Cattaneo Francesco (900411) Carpio Herreros Marco (899802)

Il progetto si trova sulla repository [Repository DCT](#).

Introduzione

In questo progetto abbiamo implementato la DCT2, per la compressione delle immagini JPEG in toni di grigio, in C#. In particolare, il progetto è diviso in due parti, le quali verranno spiegate nei capitoli successivi:

- Prima parte: Confronto del tempo di esecuzione tra la DCT implementata nel progetto, e la DCT implementata nella libreria Accord
- Seconda parte: Creazione di un'interfaccia utente in Avalonia che permette all'utente di caricare un'immagine bitmap in toni di grigio, e comprimerla in base ai valori dati

Classi

Classe DCT2

Questa classe implementa tutti i calcoli necessari per la DCT e la IDCT. Ognuno di questi metodi funziona su matrici di tipo double. È strutturata nella seguente maniera:

- `public static List<List<double>> matrice(int n):` Creazione di una matrice $n \times n$ riempita con numeri casuali
- `public static List<List<double>> D_matrix(int n):` Restituisce la matrice dei coefficienti; ogni elemento è calcolato come $D_{i,j} = d_K \cdot \cos\left(\frac{i \cdot \pi \cdot (2j+1)}{2n}\right)$, con
$$\begin{cases} \text{se } i = 0 \rightarrow d_K = \frac{1}{\sqrt{n}} \\ \text{se } i > 0 \rightarrow d_K = \sqrt{\frac{2}{n}} \end{cases}$$
- `public static List<double> cVector(List<List<double>> D, List<double> f):` Calcola il vettore dei coefficienti della DCT prendendo la matrice di trasformazione D e applicandola ad un vettore di dati di input f. Ogni coefficiente viene calcolato come $C_i = \sum_{j=0}^{n-1} D_{i,j} \cdot f_j$
- `public static List<List<double>> dct(List<List<double>> matrice):` Calcola la DCT su una matrice quadrata sfruttando la separabilità, ovvero applicandola prima alle righe, e poi alle colonne.
- `public static List<List<double>> idct (List<List<double>> matrice):` Prende la matrice dei coefficienti di frequenza e ricostruisce la matrice originale.

Classe Funzioni

Questa classe implementa varie funzioni di supporto e funzioni per applicare la DCT su un'immagine. Sono state implementate due funzioni di conversione dei tipi di matrici, per adeguare le richieste del linguaggio.

- `public static double[,] convertitore(List<List<double>> list):` Converte una matrice di tipo `List<List<double>>` in `double[,]`
- `public static List<List<double>> convertitore(double[,] matrice):` Converte una matrice di tipo `double[,]` in `List<List<double>>`
- `public static Risultato calcoloDCT(int n):` A partire dalla matrice con numeri casuali di dimensione $n \times n$, calcola il tempo di esecuzione della DCT implementata in questa libreria, e quella nella libreria Accord.
- `public static double[,] convertiImmagineMatrice(Bitmap immagine):` Converte un'immagine in una matrice di numeri double contenente i valori di luminosità dei pixel
- `public static double[,] convertiImmaginiBlocchi(double[,] matrice, int F, int d, CancellationToken token):` Esegue la DCT partendo dalla matrice dei pixel dell'immagine, dividendola in blocchi di dimensione $F \times F$, eliminando frequenze più alte in base ad una soglia d , e applicando la IDCT per ricomporre l'immagine finale.
- `public static void stampaMatriceDebug(double[,] matrice):` Stampa la matrice, passata come parametro, nella console di debug.
- `public static WriteableBitmap conversioneMatriceBitmap(double[,] matrice):` Converte una matrice di pixel in un'immagine bitmap, visualizzabile in un controllo di tipo Image

Classe Risultato

Questa classe confronta i tempi di esecuzione della DCT fatta in casa con la DCT della libreria Accord. Include tre parametri: dimensione della matrice, tempo di esecuzione della matrice fatta in casa e tempo di esecuzione della libreria Accord. Questa classe verrà usata per la realizzazione del grafico della prima parte del progetto.

Classe MainWindow

Questa classe rappresenta la parte di logica dell'interfaccia utente, e non implementa nessun calcolo pertinente alla DCT.

In particolare, la funzione `private async void calcoloTest(object sender, RoutedEventArgs e);` implementa i test richiesti dal pdf con i seguenti risultati:

$$\begin{bmatrix} 1119 & 44 & 76 & -139 & 3 & 122 & 195 & -102 \\ 77 & 115 & -22 & 41 & 9 & 99 & 138 & 11 \\ 45 & -63 & 112 & -76 & 124 & 96 & -40 & 59 \\ -70 & -40 & -23 & -77 & 27 & -37 & 66 & 125 \\ -109 & -43 & -56 & 8 & 30 & -29 & 2 & -94 \\ -5 & 57 & 173 & -35 & 32 & 33 & -58 & 19 \\ 79 & -65 & 119 & -15 & -137 & -31 & -105 & 40 \\ 20 & -78 & 1 & -72 & -22 & 81 & 64 & 6 \end{bmatrix}$$

[4,02e+02 6,60e+00 1,09e+02 -1,13e+02 6,54e+01 1,22e+02 1,17e+02 2,88e+01]

Parte 1

È stato effettuato il confronto di tempo di esecuzione tra la DCT fatta in casa e quella della libreria Accord.

Il confronto si è svolto su matrici di dimensione 8 x 8, 16 x 16, 32 x 32, 64 x 64, 128 x 128, 256 x 256, 1024 x 1024.

I tempi vengono convertiti in scala logaritmica in base 10, poiché i tempi per le matrici piccole, se non ci fosse questa conversione, sembrerebbero tutti schiacciati sullo 0 rispetto agli altri.

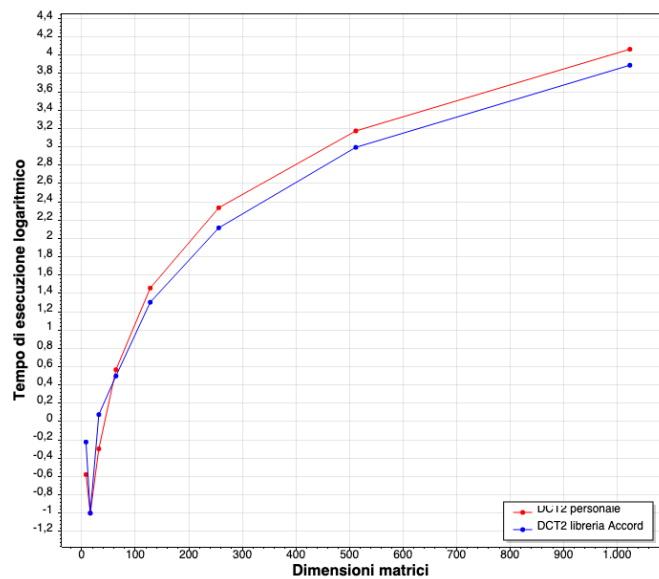
I risultati ottenuti sono che a dimensione piccole (fino a matrici di dimensione 32 x 32) la libreria Accord è più lenta rispetto a quella implementata. Per dimensioni superiori il risultato si inverte.

Il metodo richiamato con Accord è `CosineTransform.DCT(matriceAccord)`, ovvero l'implementazione della DCT-II. È stata scelta Accord come libreria poiché è la libreria standard open-source in .NET per queste operazioni.

Per la creazione del grafico è stato usato il grafico AvaPlot, configurato a partire all'avvio dell'applicazione nella funzione `MainWindow_Loaded` della classe `MainWindow`; per la versione finale dell'applicazione, tale funzione è stata commentata per permettere all'utente di usare solo la DCT.

Di seguito, il dettaglio dei risultati:

Dim	Tempo DCT (in s)	Tempo Accord (in s)
8x8	0,2636	0,5976
16x16	0,0725	0,0665
32x32	0,5034	1,1922
64x64	3,687	3,1459
128x128	28,669	20,0802
256x256	215,9433	130,2975
512x512	1491,6578	987,0931
1024x1024	11637,2465	7781,8325



Parte 2

È stata creata un'interfaccia in modo che l'utente possa scegliere dal filesystem un'immagine .bmp in toni di grigio, per poi applicarci su di essa la DCT in base ai valori di F e d dati dall'utente. Per la creazione di essa è stato utilizzato il framework Avalonia ed il linguaggio C#.

Si è scelto tale framework perché permette la creazione di app native cross-platform (Windows, macOS, Linux) senza dover riscrivere il codice, e senza usare quantità ingenti di risorse hardware.

Struttura dell'interfaccia

Attraverso l'interfaccia l'utente può eseguire diverse operazioni:

- Modificare la variabile F: un intero che rappresenta l'ampiezza dei macro-blocchi dove verrà effettuata la DCT2.
- Modificare la variabile d: un intero, compreso tra 0 e $(2F-2)$ che rappresenta la soglia di taglio delle frequenze.
- Utilizzare il bottone "Selezione immagine" per scegliere solo immagini .bmp.
- Utilizzare il bottone play/stop per interrompere o iniziare la DCT sull'immagine.
- Utilizzare i bottoni radio "Fill" e "Stretch" per scegliere la modalità di visualizzazione delle immagini.
- Utilizzare il bottone "ingranaggio" per eseguire i test sui numeri richiesti nel pdf.
- Utilizzare lo slider per lo zoom delle immagini.

Struttura del codice

Usando il framework Avalonia ciascuna finestra è divisa in due file: un file .cs, per la logica, e un file .axaml, per gestire la parte grafica.

File AXAML

Il file segue una struttura gerarchica: Prima è definita la status bar, poi i riquadri delle immagini e, infine, la finestra di controllo.

La status bar è ancorata al fondo della finestra dell'app, e mostra informazioni utili o errori all'utente in tempo reale.

Nel riquadro a sinistra viene caricata l'immagine originale, mentre nel riquadro a destra viene mostrata l'esecuzione della DCT su di essa. Inoltre, trascinando la barra centrale, che divide i due riquadri, è possibile ridimensionare i due riquadri.

La finestra di controllo è sempre in primo piano rispetto alle immagini per poter eseguire le operazioni descritte prudentemente.

Inoltre, il codice presenta tutte queste particolarità:

- Grazie all'uso di StackPanel e Grid l'interfaccia si ridimensiona in automatico.
- L'interfaccia si adatta automaticamente alle impostazioni di aspetto di sistema.
- Le immagini sono raggruppate ciascuna in uno ScrollViewer e un LayoutTransformControl, che permette lo zoom all'interno di esse
- È possibile utilizzare scorciatoie di tastiera per:
 - o Scegliere una nuova immagine (CTRL + O)
 - o Iniziare/fermare la DCT (CTRL + D)
- Se si sceglie un valore delle variabili oltre il massimo consentito viene inserito di default il massimo possibile di esse, mentre se non si scrive nessun valore allora viene inserito il minimo possibile di esse.
- È possibile fare lo zoom attraverso le gesture del touchscreen/touchpad (solo su macOS), o con lo slider nella finestra di controllo. Lo zoom e la posizione all'interno delle immagini sono uguali per entrambi

File CS

Tale file rappresenta la logica per la finestra. Le parti di codice commentate rappresentano il codice servito per la prima parte del progetto.

Per permettere che l'interfaccia non si blocchi durante l'esecuzione della DCT, le operazioni matematiche sono eseguite su thread diversi rispetto al thread UI.

Le funzioni all'interno sono le seguenti:

- `public MainWindow()`: costruttore della finestra; inizializza gli stati dei bottoni, e lo zoom via gesture sull'immagine
- `private async void disabilitaUI(), abilitaUI()`: durante l'esecuzione della DCT, la possibilità di modificare i valori viene disabilitata, per poi essere abilitata quando la si interrompe, o finisce
- `private async void bottoneStop(object sender, RoutedEventArgs e)`: permette l'inizio e l'interruzione della DCT. Questo è possibile perché la DCT viene eseguita su un thread separato, dunque è possibile interromperlo a piacere
- `private async void proporzioneImmagine(object sender, RoutedEventArgs e)`: permette di cambiare la visualizzazione delle immagini; Fill mantiene la proporzione originale, mentre Stretch riempie il contenitore con l'immagine
- `private async void caricaImmagine()`: apre il selettore di file nativo per far scegliere all'utente un'immagine .bmp
- `public async void valoriDCT()`: esegue la DCT, dopo che l'utente ha selezionato un'immagine. Disabilita l'interfaccia temporaneamente, mostra all'utente lo stato dell'esecuzione, ed infine visualizza l'immagine risultante
- `private void cambioValoreF(object? Sender, NumericUpDownValueChangedEventArgs e)`: controlla se i valori di F e d sono all'interno dei range
- `private void Immagini_Zoom(object sender, Avalonia.Input.PointerDeltaEventArgs e)`: calcola il fattore di zoom generato con le gesture, o con lo slider
- `private void ScrollOriginale_posizione(object? sender, Avalonia.Controls.ScrollChangedEventArgs e)/ScrollDCT_posizione`: sincronizza la posizione all'interno delle due immagini, dopo aver effettuato lo zoom
- `private async void calcoloTest(object sender, RoutedEventArgs e)`: dopo aver creato la matrice e l'array, esegue la DCT e la DCT 1D su ciascuna, stampando i risultati nella console di debug

Durante lo sviluppo si è visto che l'app, ad ogni cambio immagine, non liberava la memoria, dunque portando a memory leak di dimensioni elevate, specie con le immagini più grandi.

Sono state applicate alcune strategie per liberarla:

- Quando una nuova immagine viene caricata, la vecchia immagine originale ed elaborata viene distrutta.
- Quando la DCT viene interrotta ogni dato temporaneo viene distrutto.

- Nelle funzioni della DCT dove abbiamo usato puntatori, per evitare memory leak la memoria è stata gestita manualmente.

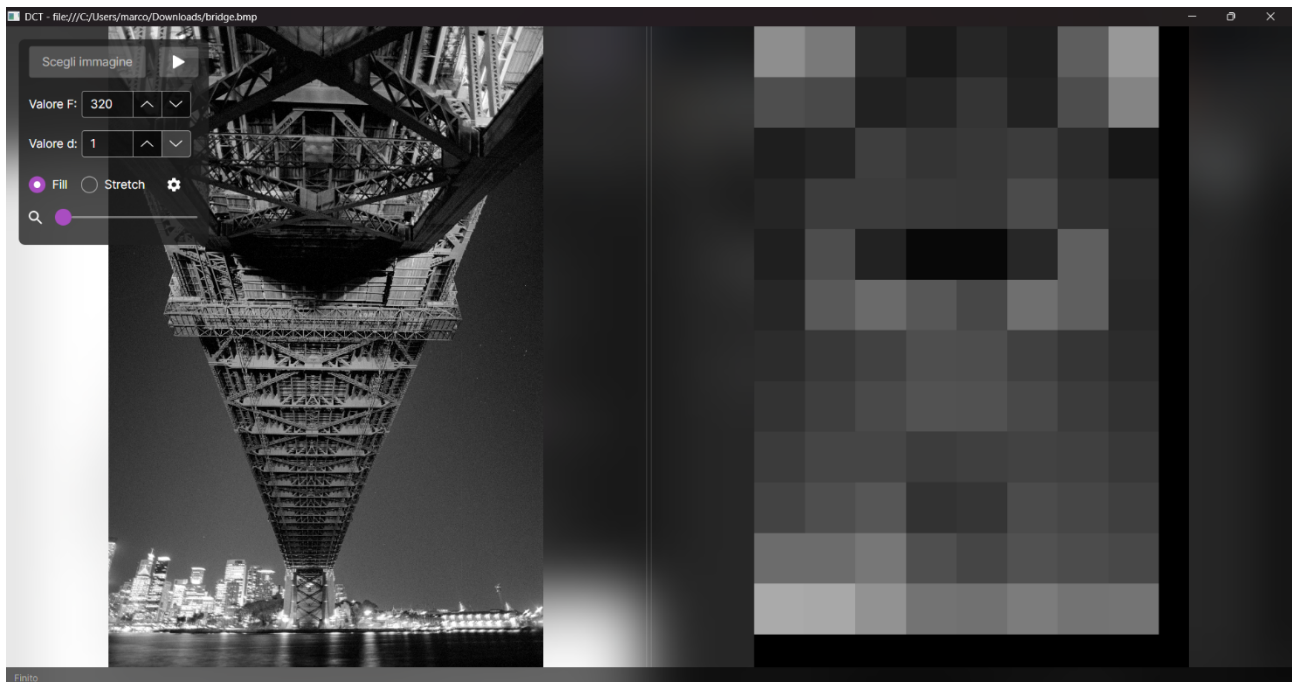
Risultati

Esempio 1: compressione a blocchi grandi; prima $d=1$, poi $d=20$



Come si può vedere, quando il valore di d si avvicina di più al valore di F , il risultato che si ottiene dalla DCT è sempre più simile all'immagine originale.

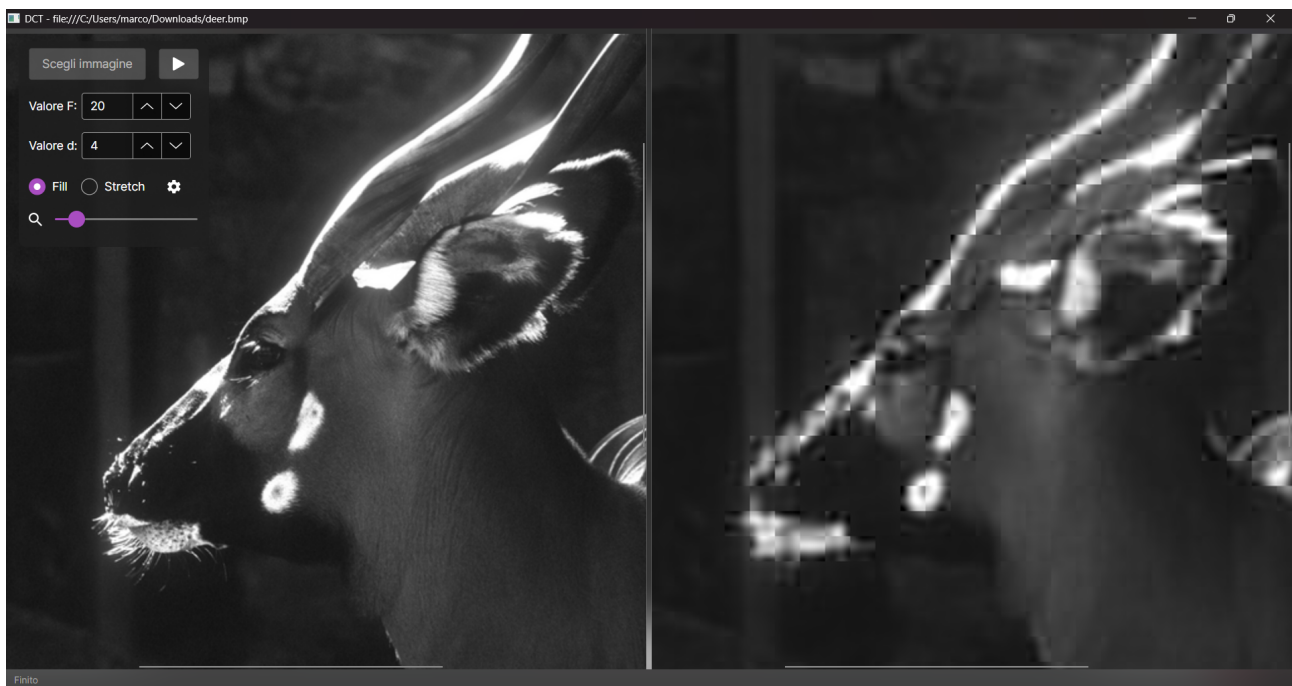
Esempio 2: compressione di immagini a blocchi molto grandi



Il progetto supporta la divisione in blocchi con valori molto elevati, seppure questi, a volte, sono inutili dal punto di vista pratico. Visto che le dimensioni dell'immagine (2000x3000 pixel) non sono divisibili per la grandezza del blocco ($F = 320$), i blocchi esterni "tagliati" vengono riempiti con pixel neri.

Inoltre, le immagini sono mostrate in modalità Fill, mantenendo le proporzioni originali.

Esempio 3: Utilizzo dello zoom



Grazie allo zoom è possibile visualizzare i blocchi in modo più evidente. In questo esempio, la compressione ha mantenuto qualche dettaglio, abbastanza per creare una copia lontanamente simile all'originale, ma non abbastanza per non vedere la divisione in blocchi.